Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИУ «Информатика и системы управления»

КАФЕДРА ИУ-7 «Программное обеспечение ЭВМ и информационные технологии»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К КУРСОВОЙ РАБОТЕ

НА ТЕМУ:

«Загружаемый модуль ядра для визуализации использования системных ресурсов с помощью клавиатуры»

Студент группы ИУ7-73Б _____ Паламарчук А.Н.
(Подпись, дата) (Фамилия И.О.)

Руководитель _____ Рязанова Н.Ю.
(Подпись, дата) (Фамилия И.О.)

2025 г.

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

УТВЕРЖДАЮ
Заведующий кафедрой ИУ7
_____ И. В. Рудаков
«__» сентября 2025 г.

З А Д А Н И Е
на выполнение курсовой работы

по дисциплине **«Операционные системы»**

Студент группы **ИУ7-73Б** **Паламарчук Андрей Николаевич**

Тема курсовой работы **Загружаемый модуль ядра для визуализации использования системных ресурсов с помощью клавиатуры**

Направленность КР (учебная, исследовательская, практическая, др.) **учебная**

Источник тематики (кафедра, предприятие, НИР) **кафедра**

График выполнения КР: 25% к 5 нед., 50% к 8 нед., 75% к 11 нед., 100% к 15 нед.

Задание:

Разработать драйвер который преобразует внешнюю клавиатуру в устройство световой индикации отображающее параметры системы. Для визуализации должны быть реализованы 3 шкалы по буквенным рядам клавиатуры.

Оформление курсовой работы:

Расчетно-пояснительная записка объемом 30-40 страниц формата А4. Расчетно-пояснительная записка должна содержать постановку задачи, введение, аналитический раздел, конструкторский раздел, технологический раздел, исследователький раздел, заключение, список литературы и приложения.

Дата выдачи задания «22» октября 2025 г.

Руководитель курсовой работы _____ _Н. Ю. Рязанова_
(Подпись, дата) (И.О.Фамилия)

Студент _____ _А.Н.Паламарчук
(Подпись, дата) (И.О.Фамилия)

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 4

1. Аналитический раздел 5

1.1. Постановка задачи 5

1.2. Драйвер 5

1.3. USB-подсистема ядра Linux 6

1.4. Анализ параметров системы 9

1.5. Эксперимент для анализа протокола обмена данными 10

1.6. Выводы 11

2. Конструкторский раздел 12

2.1. IDEF0-диаграмма 12

2.2. Алгоритм функционирования драйвера 13

2.3. Алгоритм обновления подсветки 14

2.4. Алгоритм формирования цветового пакета 15

3. Технологический раздел 16

3.1. Выбор средств реализации 16

3.2. Функциональность разработанного ПО 16

3.3. Объявление структуры данных драйвера 16

3.4. Реализация взаимодействия с подсистемой USB 17

3.5. Сборка и установка модуля 18

4. Исследовательский раздел 19

ЗАКЛЮЧЕНИЕ 20

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 21

ПРИЛОЖЕНИЕ 22

ВВЕДЕНИЕ

Возможность отслеживания состояния вычислительной системы напрямую влияет на пользовательский опыт работы. Стандартные программные средства мониторинга занимают экранное пространство и могут быть перекрыты окнами используемых приложений.

Клавиатура Aula F75 с настраиваемой подсветкой предоставляет возможности для создания средства индикации, независимого от графического интерфейса. Поэтому разработка ПО, позволяющего использовать клавиатуру для индикации, является актуальной задачей, направленной на расширение функциональности устройства и улучшение пользовательского опыта.

Данная работа посвящена разработке ПО для визуализации использования системных ресурсов с помощью клавиатуры.

1. Аналитический раздел

1.1. Постановка задачи

В соответствии с заданием на КР необходимо решить задачу визуализации параметров системы, используя светодиодную подсветку клавиатуры.

Для решения поставленной задачи необходимо:

- провести анализ параметров системы для визуализации наиболее показательных;
- разработать эксперимент позволяющий получить информацию о протоколе обмена данными для клавиатуры Aula F75;
- разработать алгоритмы, необходимые для реализации ПО;
- разработать ПО, предоставляющее требуемую функциональность.

1.2. Драйвер

В современных ядрах Linux изменение функциональности внешних устройств возможно с помощью драйверов, представляющих собой загружаемые модули ядра. В Linux имеется три типа драйверов:

1. драйверы первого типа являются частью программного кода ядра. Соответствующие устройства автоматически обнаруживаются системой и становятся доступны для приложений;
2. драйверы второго типа представлены загружаемыми модулями ядра. Они оформлены в виде отдельных файлов. Могут быть загружены и выгружены из памяти во время работы системы. Использование модулей обеспечивает большую гибкость, так как каждый драйвер может быть переконфигурирован без перезагрузки системы;
3. в драйверах третьего типа программный код драйвера поделен между ядром и специальной утилитой, предназначенной для управления данным устройством.

1.3. USB-подсистема ядра Linux

USB-подсистема ядра Linux предоставляет унифицированный интерфейс для работы с устройствами, подключёнными к USB-шине (Universal Serial Bus).

Существует 4 типа передачи данных по USB-шине:

1. **control** — двунаправленная передача с гарантированной доставкой. Предназначена для обмена с устройством коротких пакетов типа «вопрос–ответ». Используется системным ПО для отправки определенных общих команд на USB-устройство и позволяет прочитать информацию об устройстве: коды производителя — vendor и модели — product.
2. **interrupt** — передача коротких пакетов, требующих гарантированного времени обслуживания. Применяется в устройствах, генерирующих спонтанные события, например, при вводе символов с клавиатуры или для передачи данных о перемещении мыши.
3. **bulk** — сплошная поточная передача данных. Используется устройствами, оправляющими и принимающими большое количество данных, но не имеющих определенную пропускную способность. Время задержки не определено. Пакеты типа bulk имеют самый низкий приоритет, то есть передаются в последнюю очередь и занимают всю свободную полосу пропускания шины.
4. **isochronous** — непрерывная передача данных. Обеспечивает гарантированную пропускную способность: N пакетов за 1 период шины и определенные задержки. Передача осуществляется без подтверждения приема.

В разрабатываемом драйвере для управления подсветкой используется control. Обусловлено тем, что команды смены цвета требуют гарантированной доставки пакетов настройки.

Каждый USB-драйвер должен определить:

—таблицу идентификаторов устройств (`struct usb_device_id`) (листинг:1.1), связывающую драйвер с конкретными vendorID и productID;

—структуру драйвера (`usb_driver`) с callback-функциями (`probe`, `disconnect`, `id_table`), представленную в листинге 1.2;

—функции управления жизненным циклом модуля, которые регистрируются с помощью макросов `module_init` и `module_exit`.

Листинг1.1 — Структура `usb_device_id`

```
struct usb_device_id {
    /* which fields to match against? */
    __u16 match_flags;

    /* Used for product specific matches; range is inclusive */
    __u16 idVendor;
    __u16 idProduct;
    ...
};
```

—`match_flags` — битовая маска, определяющая, какие из других полей используются для сопоставления с новыми устройствами. Обычно задается макросом `USB_DEVICE_*`(), который инициализирует остальные необходимые поля;

—`idVendor` — идентификатор производителя USB для устройства;

—`idProduct` — идентификатор продукта, присвоенный производителем.

Листинг1.2 — Структура `usb_driver`

```
struct usb_driver {
    const char *name;
    int (*probe) (struct usb_interface *intf ,
                const struct usb_device_id *id);
    void (*disconnect) (struct usb_interface *intf);
    ...
    const struct usb_device_id *id_table;
    ...
};
```

Основной структурой для обмена данными по шине USB является `struct urb` (USB Request Block), представленная на листинге1.3. Структура URB содержит параметры для описания USB-запроса и применяется для всех типов передачи данных.

Листинг1.3 — Структура urb

```
struct urb {
    struct usb_device *dev;
    int status;
    unsigned int transfer_flags;
    void *transfer_buffer;
    u32 actual_length;
    void *context;
    ...
};
```

Основные поля:

- `struct usb_device *dev` — указатель на объект USB-устройства, к которому относится данный запрос. Определяет физическое устройство и хост-контроллер для передачи.
- `int status` — содержит статус завершения обработки запроса. Значение 0 означает успех. Отрицательные значения соответствуют кодам ошибок ядра Linux.
- `unsigned int transfer_flags` — набор флагов, управляющих деталями передачи. Например:
  - `URB_SHORT_NOT_OK` — интерпретировать короткий пакет как ошибку;
  - `URB_NO_TRANSFER_DMA_MAP` — буфер уже отображён для DMA.
  - `URB_ZERO_PACKET` — для завершения передачи отправить нулевой пакет.
- `void *transfer_buffer` — указатель на буфер данных для передачи или для приема данных. Этот буфер используется на этапе передачи данных при передаче управления.

—`u32 actual_length` — после завершения передачи в это поле записывается фактическое количество байт, переданных или принятых через USB. Для изохронных передач это сумма длин по всем пакетам.

—`void *context` — указатель на произвольные данные драйвера (контекст). Для использования в функциях завершения.

1.4. Анализ параметров системы

Load Average — показатель среднего количества процессов, находящихся в состоянии исполнения или ожидания ресурсов (CPU/Disk). Превышение значения относительно числа ядер свидетельствует о перегрузке системы и позволяет диагностировать зависания, вызванные как вычислениями, так и медленным вводом-выводом [1].

PID Usage — отношение текущего количества процессов к системному лимиту. Мониторинг позволяет предотвратить состояние отказа в обслуживании, возникающее при неконтролируемом порождении процессов (например, fork-бомбы), когда запуск новых задач становится невозможным.

Context Switches — частота смены выполняемых потоков на процессоре в единицу времени. Аномально высокие значения при низкой полезной загрузке CPU сигнализируют о неэффективности программного обеспечения или аппаратных сбоях, приводящих к высоким накладным расходам на обслуживание самой ОС [2].

Major Page Faults — количество обращений к страницам памяти, выгруженным в раздел подкачки, требующих чтения с диска. Индикатор состояния, при котором система тратит больше времени на перемещение данных между диском и памятью, чем на полезную работу.

Memory Pressure Stall Information — параметр, характеризующий время простоя процессов из-за ожидания выделения памяти. PSI позволяет определить моменты, когда дефицит ресурсов начинает напрямую влиять на отзывчивость системы[3].

Dirty Pages — объем данных в оперативной памяти, ожидающих записи на физический носитель. Накопление объема таких страниц приводит к принудительной блокировке ввода-вывода для сброса буферов, что для пользователя выглядит как полное зависание интерфейса системы [4].

File Descriptors — количество используемых файловых дескрипторов относительно лимита ядра. Исчерпание этого ресурса ведет к аварийному завершению.

Inode Usage — степень заполненности таблицы файловой системы. Позволяет отслеживать ситуации (актуальные для серверов с множеством мелких файлов), когда создание новых файлов невозможно, несмотря на наличие свободного дискового пространства.

Conntrack Table — количество отслеживаемых сетевых соединений. Переполнение таблицы (например, при P2P-активности или DDoS-атаках) приводит к отбрасыванию пакетов новых соединений[5].

1.5. Эксперимент для анализа протокола обмена данными

Ввиду отсутствия официальной документации для протокола управления клавиатурой Aula F75, необходимо экспериментальным путем установить формат команд, путем перехвата и анализ трафика, между устройством и оригинальным ПО. Для проведения эксперимента был использован Wireshark в связке с usbmon.

В ходе эксперимента рассматривались следующие тестовые случаи:

- установка цвета всех клавиш для выявления неизменяемой части пакета и формата кодирования цвета;
- установление цвета одной клавиши для определения адресации и поиска смещения соответствующего байта в массиве данных;
- окрашивание рядов клавиш для определения границ блоков данных, отвечающих за различные цветовые каналы (Red, Green, Blue).

Анализ заключался в побайтовом сравнении полученных дампов. Путем сопоставления изменений в отправляемых данных с визуальными изменениями на устройстве выявлялись закономерности формирования пакетов.

В результате проведенного эксперимента было установлено, что данные цветов передаются не последовательно для каждой клавиши, а блоками по цветовым каналам. Также выявлена особенность контроллера — необходимость отправки двух пакетов перед началом управления подсветкой, без которой устройство игнорирует последующие команды.

1.6. Выводы

В результате проведенного анализа:

- для передачи данных между устройством и драйвером будет использоваться тип передачи control и `struct urb`;
- для визуализации выбраны параметры: Load Average, Memory PSI и Dirty Pages;
- определена методика экспериментального анализа протокола управления клавиатурой.

2. Конструкторский раздел

2.1. IDEF0-диаграмма

На рисунке 2.1 представлена диаграмма IDEF0, описывающая ПО.

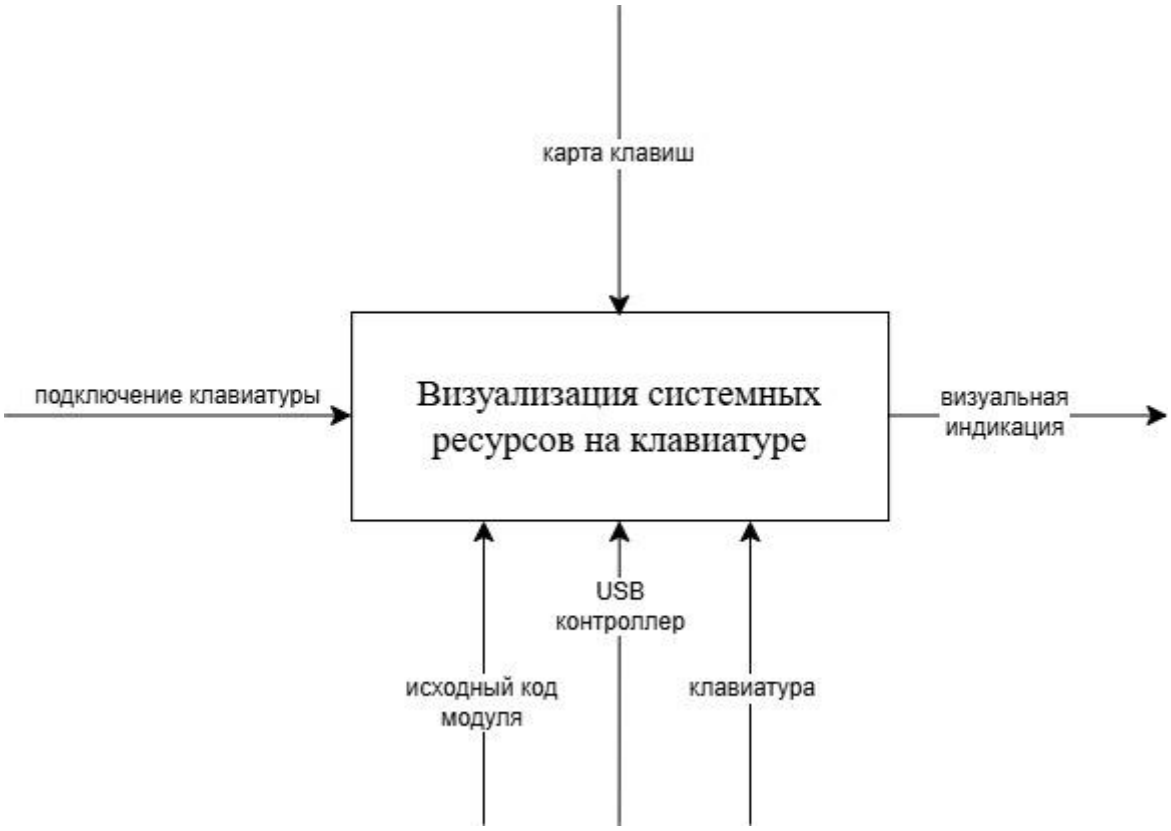

Рисунок 2.1 — IDEF0 A0

2.2. Алгоритм функционирования драйвера

На рисунке 2.2 представлена схема алгоритма функционирования драйвера.

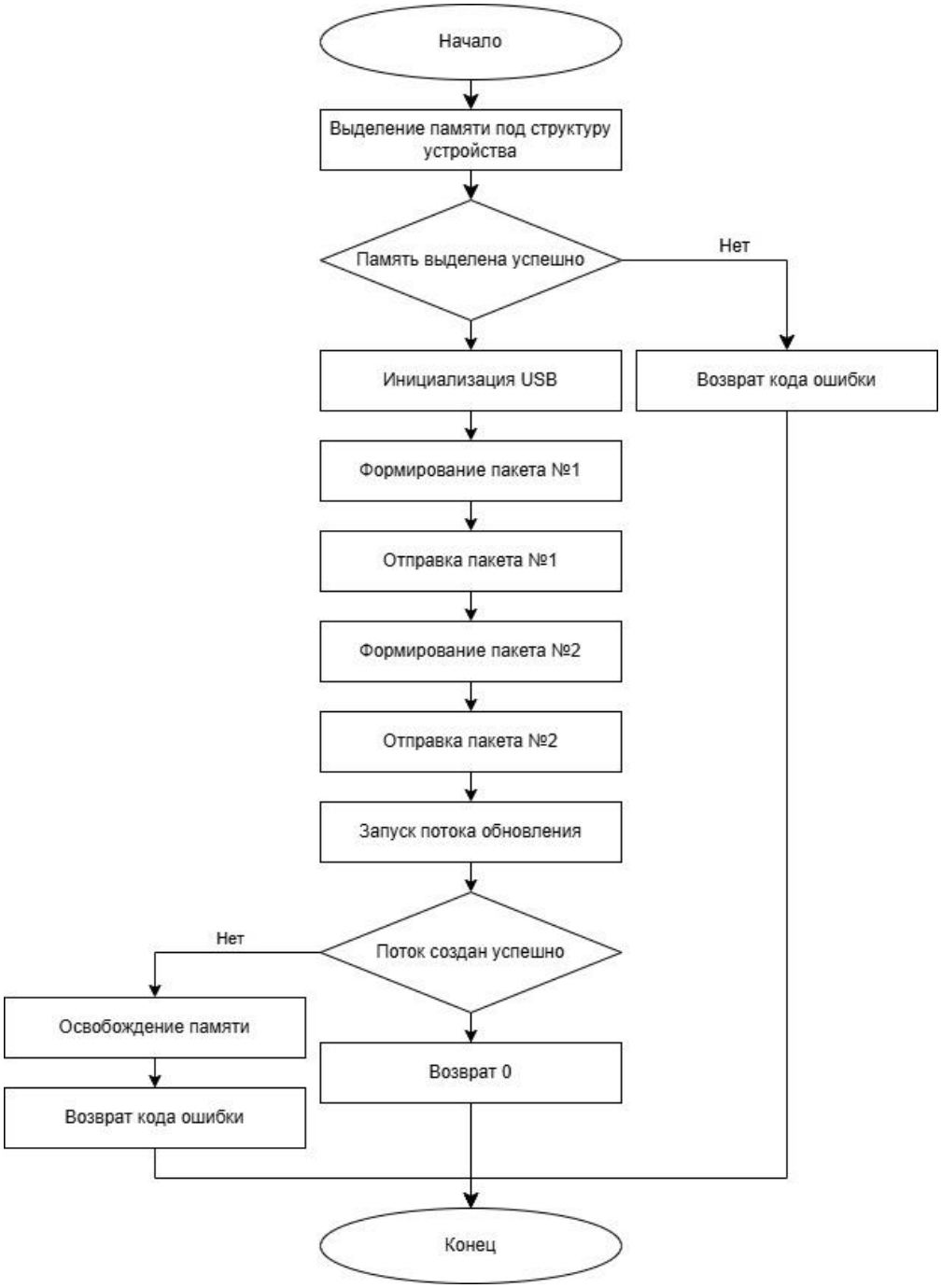

Рисунок 1.2 — Схема алгоритма функционирования драйвера

2.3. Алгоритм обновления подсветки

На рисунке 2.3 представлена схема алгоритма обновления подсветки.

Рисунок 2.2 — Схема алгоритма обновления подсветки

2.4. Алгоритм формирования цветового пакета

На рисунке 2.4 представлена схема алгоритма формирования цветового пакета.

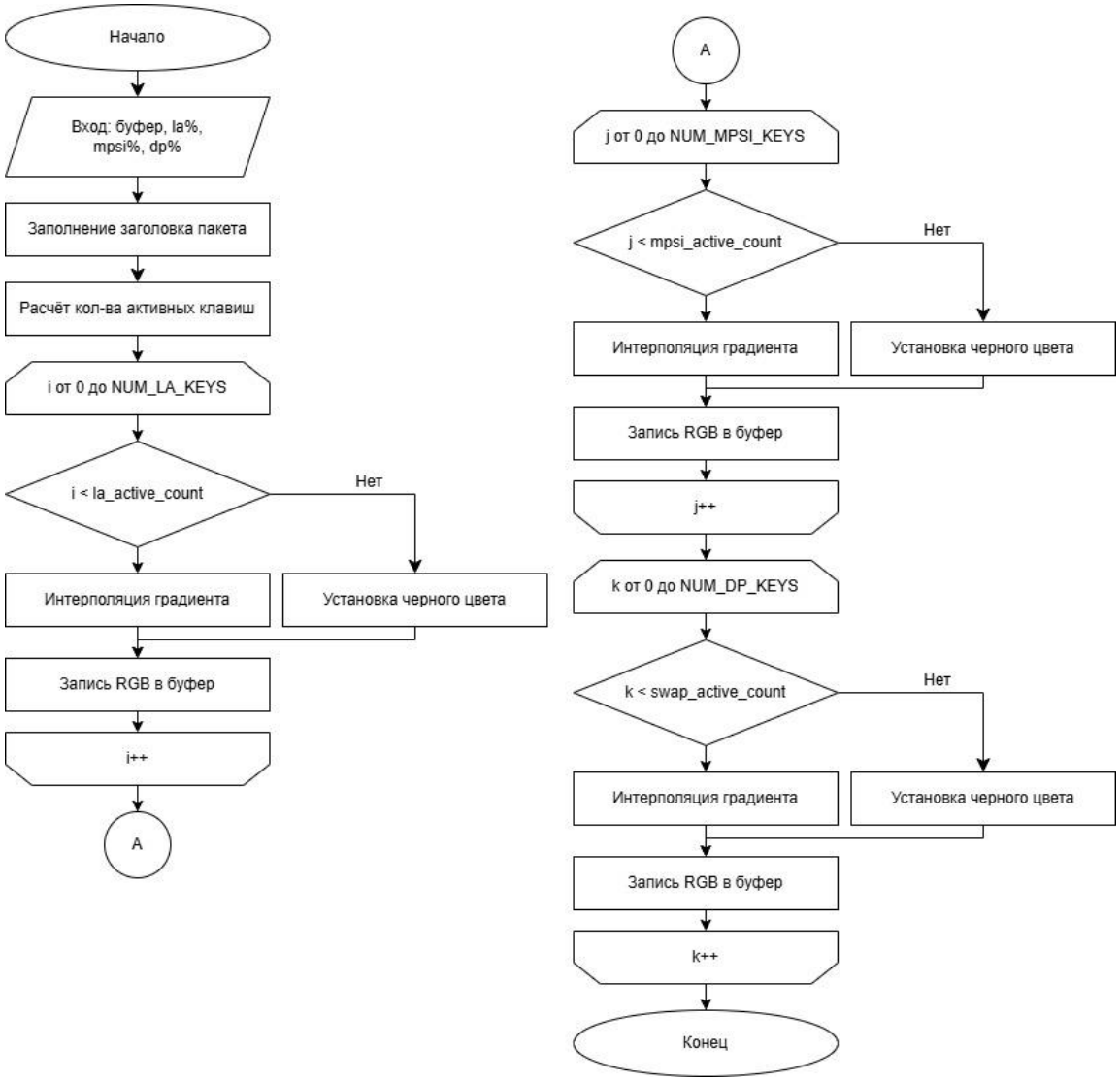

Рисунок 2.3 — Схема алгоритма формирования цветового пакета

3. Технологический раздел

3.1. Выбор средств реализации

В качестве языка программирования был выбран язык C. На этом языке реализованы все модули ядра и драйверы операционной системы Linux. В качестве компилятора был выбран gcc, в качестве среды разработки был выбран редактор VS Code. Для проведения эксперимента использован Wireshark.

3.2. Функциональность разработанного ПО

Разработанный драйвер преобразует клавиатуру Aula F75 в устройство световой индикации. Основная функциональность включает:

—перевод клавиатуры в режим настройки посредством отправки последовательности пакетов;

—циклический сбор параметров: Load Average, Memory PSI и Dirty Pages;

—преобразование значений параметров в цветовую схему для трех шкал;

—обновление окраски светодиодной подсветки клавиатуры.

Драйвер работает в фоновом режиме, используя поток ядра (kthread), что обеспечивает обновление данных без блокировки основных процессов системы.

3.3. Объявление структуры данных драйвера

Для хранения для хранения необходимых данных использована структура aula_kbd, представленная в листинге 3.1.

Листинг 3.1 — Структура aula_kbd

```
struct aula_kbd {

    struct usb_device *udev;

    struct task_struct *update_thread;

    unsigned char *packet_1;

    unsigned char *packet_2;

    unsigned char *color_packet;
```

```
    int last_la_level;

    int last_mp_level;

    int last_dp_level;


    unsigned long prev_pgmajfault;

};
```

Структура содержит указатель на USB-устройство, дескриптор потока обновления, буферы для пакетов данных и переменные для хранения предыдущих состояний параметров.

3.4. Реализация взаимодействия с подсистемой USB

Реализована функция `send_packet`, которая для отправки управляющих сообщений использует функцию `usb_control_msg`, представленна на листинге 3.2.

```
static void send_packet(struct usb_device *udev, unsigned char *data) {

    int ret = usb_control_msg(

        udev,

        usb_sndctrlpipe(udev, 0),

        0x09,

        0x21,

        0x0306,

        1,

        data,

        PACKET_BUFFER_SIZE,

        500

    );

    // Обработка ошибок

}
```

Параметры вызова были получены в ходе эксперимента:

—`usb_sndctrlpipe(udev, 0)` — указывает, что данные отправляются на нулевую контрольную точку;

—0x09 (`bRequest`) — стандартный запрос `SET_REPORT` протокола HID;

—0x21 (`bmRequestType`) — тип запроса;

—0x0306 (`wValue`) — старший байт 03 означает тип отчета (Feature Report), младший байт 06 — идентификатор отчета (Report ID);

—1 (`wIndex`) — индекс интерфейса, к которому адресуется запрос.

3.5. Сборка и установка модуля

Для автоматизации сборки используется Make. Реализованы следующие сценарии:

— default — компиляция модуля aula_monitor.ko;

— load/unload — загрузка и выгрузка модуля;

— unbind — принудительное освобождение интерфейса клавиатуры от стандартного драйвера usbhid через sysfs;

— info — lsusb -t для отображения дерева подключенных USB-устройств;

— test — сценарий для быстрой проверки работоспособности модуля;

— reload — полная пересборка(выгружает, компилирует, загружает).

4. Исследовательский раздел

Для проверки соответствия визуальной индикации реальному состоянию системы применялось нагрузочное тестирование с мониторингом через эталонные утилиты (htop, vmstat, uptime, /proc/meminfo).

Проверка индикации Load Average проводилась с помощью утилиты stress–ng, создающей нагрузку, превышающую количество ядер процессора. Индикаторная шкала на клавиатуре корректно демонстрировала рост очереди планировщика, заполняя шкалу в соответствии со значением, отображаемым командой uptime.

Индикация Memory Pressure тестировалась путем запуска скрипта, принудительно вызывающего активный обмен данными с разделом подкачки. Реакция индикаторной шкалы коррелировала с данными утилиты vmstat.

Для тестирования индикатора Dirty Pages моделировалась ситуация накопления несохраненных данных путем копирования большого файла на медленный USB-накопитель с помощью утилиты dd. Индикаторная шкала корректно отображала рост объема грязных страниц, коррелировала с выводом /proc/meminfo.

Оценка накладных расходов показала, что поток драйвера в состоянии покоя потребляет менее 0.1% одного ядра CPU. В моменты перерисовки потребление кратковременно возрастает до 0.3-0.5%. Нагрузка на шину USB составляет порядка 1.5-2 КБ/с, что является менее 0.001% от пропускной способности для шины USB 2.0.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы были проанализированы параметры системы, особенности USB-подсистемы и определен протокол взаимодействия с клавиатурой Aula F75.

Было определено, что для выполнения поставленной задачи необходимо реализовать драйвер в виде загружаемого модуля ядра, используя тип передачи control для отправки данных.

В результате экспериментального анализа протокола была выявлена последовательность пакетов для перевода устройства в режим настройки и пакет для изменения подсветки.

Было проведено исследование разработанного драйвера, показавшее, что драйвер корректно отображает выбранные параметры системы и не оказывает значительного влияния на производительность.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Corbet J., Rubini A., Kroah-Hartman G. Linux Device Drivers. 3rd Edition. — Sebastopol: O'Reilly Media, 2005. — 636 p.
2. Love R. Linux Kernel Development. 3rd Edition. — Upper Saddle River: Addison-Wesley Professional, 2010. — 496 p.
3. Mauerer W. Professional Linux Kernel Architecture. — Indianapolis: Wrox, 2008. — 1368 p.
4. Axelson J. USB Complete: The Developer's Guide. 5th Edition. — Madison: Lakeview Research, 2015. — 564 p.
5. Linux Kernel Documentation: CPU load accounting [Электронный ресурс]. — Режим доступа: https://www.kernel.org/doc/html/latest/accounting/index.html (дата обращения: 10.11.2025).
6. Linux Kernel Driver API: kthread_run [Электронный ресурс]. — Режим доступа: https://www.kernel.org/doc/html/latest/driver-api/infrastructure.html (дата обращения: 10.11.2025).
7. Kernighan B. W., Ritchie D. M. The C Programming Language. 2nd Edition. — Englewood Cliffs: Prentice Hall, 1988. — 272 p.
8. GNU Make Manual [Электронный ресурс]. — Режим доступа: https://www.gnu.org/software/make/manual/make.html (дата обращения: 10.11.2025).
9. Linux Kernel Driver API: kthread_stop [Электронный ресурс]. — Режим доступа: https://www.kernel.org/doc/html/latest/driver-api/infrastructure.html (дата обращения: 10.11.2025).
10. Universal Serial Bus Revision 2.0 specification [Электронный ресурс]. — Режим доступа: https://www.usb.org/document-library/usb-20-specification (дата обращения: 10.11.2025).

ПРИЛОЖЕНИЕ

Листинг 1 — Исходный код aula_monitor.c

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/usb.h>
#include <linux/slab.h>
#include <linux/delay.h>
#include <linux/kthread.h>
#include <linux/sched.h>
#include <linux/init.h>
#include <linux/mm.h>
#include <linux/swap.h>
#include <linux/cpumask.h>
#include <linux/kernel_stat.h>
#include <linux/vmstat.h>
#include <linux/sched/loadavg.h>

MODULE_LICENSE("GPL");
MODULE_AUTHOR("PAN");
MODULE_DESCRIPTION("Aula Critical Metrics Monitor");

#define AULA_VID 0x258a
#define AULA_PID 0x010c
#define PACKET_BUFFER_SIZE 520

#define RED_BLOCK_OFFSET   8
#define GREEN_BLOCK_OFFSET 134
#define BLUE_BLOCK_OFFSET  260

enum KeyIndex {
    KEY_PD, KEY_TILDE, KEY_TAB, KEY_CAPSLOCK, KEY_LSHIFT, KEY_LCTRL,
KEY_EXCLAMATION, KEY_1,
    KEY_Q, KEY_A, KEY_Z, KEY_WIN, KEY_F1, KEY_2, KEY_W, KEY_S, KEY_X,
KEY_LALT,
    KEY_F2, KEY_3, KEY_E, KEY_D, KEY_C, KEY_UNKNOWN_1, KEY_F3, KEY_4,
KEY_R, KEY_F,
```

```
    KEY_V, KEY_UNKNOWN_2, KEY_F4, KEY_5, KEY_T, KEY_G, KEY_B,
KEY_SPACE, KEY_F5,
    KEY_6, KEY_Y, KEY_H, KEY_N, KEY_UNKNOWN_3, KEY_F6, KEY_7, KEY_U,
KEY_J, KEY_M,
    KEY_UNKNOWN_4, KEY_F7, KEY_8, KEY_I, KEY_K, KEY_COMMA, KEY_FN,
KEY_F8, KEY_9,
    KEY_O, KEY_L, KEY_DOT, KEY_RCTRL, KEY_F9, KEY_0, KEY_P,
KEY_SEMICOLON,
    KEY_QUESTION, KEY_UNKNOWN_5, KEY_F10, KEY_MINUS, KEY_LBRACKET,
KEY_APOSTROPHE,
    KEY_RSHIFT, KEY_UNKNOWN_6, KEY_F11, KEY_EQUALS, KEY_RBRACKET,
KEY_UNKNOWN_7,
    KEY_UNKNOWN_8, KEY_LARROW, KEY_F12, KEY_BACKSPACE, KEY_BACKSLASH,
KEY_ENTER,
    KEY_UARROW, KEY_DARROW, KEY_UNKNOWN_9, KEY_DEL, KEY_PGUP, KEY_ESC,
KEY_END, KEY_RARROW,
    KEY_COUNT
};

static const enum KeyIndex load_avg_keys[] = {
    KEY_W, KEY_E, KEY_R, KEY_T, KEY_Y, KEY_U, KEY_I, KEY_O, KEY_P,
KEY_LBRACKET
};
static const enum KeyIndex mem_pressure_keys[] = {
    KEY_A, KEY_S, KEY_D, KEY_F, KEY_G, KEY_H, KEY_J, KEY_K, KEY_L,
KEY_SEMICOLON
};
static const enum KeyIndex dirty_pages_keys[] = {
    KEY_Z, KEY_X, KEY_C, KEY_V, KEY_B, KEY_N, KEY_M, KEY_COMMA,
KEY_DOT, KEY_QUESTION
};
static const enum KeyIndex border_keys[] = {
    KEY_TAB, KEY_Q, KEY_CAPSLOCK, KEY_LSHIFT, KEY_RBRACKET,
KEY_BACKSLASH, KEY_PGUP,
    KEY_APOSTROPHE, KEY_ENTER, KEY_ESC, KEY_RSHIFT, KEY_UARROW, KEY_END
};
```

```
#define NUM_LA_KEYS ARRAY_SIZE(load_avg_keys)
#define NUM_MP_KEYS ARRAY_SIZE(mem_pressure_keys)
#define NUM_DP_KEYS ARRAY_SIZE(dirty_pages_keys)
#define NUM_BORDER_KEYS ARRAY_SIZE(border_keys)

struct Color {
    unsigned char r;
    unsigned char g;
    unsigned char b;
};

struct Gradient {
    struct Color start;
    struct Color end;
};

static const struct Gradient la_gradient = { {0, 255, 0}, {255, 0, 0}
};
static const struct Gradient mp_gradient = { {0, 0, 255}, {255, 0, 255}
};
static const struct Gradient dp_gradient = { {255, 255, 0}, {255, 0, 0}
};

struct aula_kbd {
    struct usb_device *udev;
    struct task_struct *update_thread;
    unsigned char *handshake_packet_1;
    unsigned char *handshake_packet_2;
    unsigned char *color_packet;

    int last_la_level;
    int last_mp_level;
    int last_dp_level;

    unsigned long prev_pgmajfault;
```

```
};

static void set_key_color(unsigned char *buffer, enum KeyIndex key_idx,
struct Color color) {
    if (key_idx < KEY_COUNT) {
        buffer[RED_BLOCK_OFFSET + key_idx] = color.r;
        buffer[GREEN_BLOCK_OFFSET + key_idx] = color.g;
        buffer[BLUE_BLOCK_OFFSET + key_idx] = color.b;
    }
}

static struct Color lint_color(const struct Gradient *grad, int step,
int max_steps) {
    struct Color c;
    if (max_steps <= 0) return grad->start;
    if (step < 0) step = 0;
    if (step > max_steps) step = max_steps;

    c.r = grad->start.r + (((int)grad->end.r - (int)grad->start.r) *
step) / max_steps;
    c.g = grad->start.g + (((int)grad->end.g - (int)grad->start.g) *
step) / max_steps;
    c.b = grad->start.b + (((int)grad->end.b - (int)grad->start.b) *
step) / max_steps;
    return c;
}

static int get_load_average_percent(void) {
    unsigned long load = avenrun[0] >> FSHIFT;
    unsigned long load_frac = (avenrun[0] & (FIXED_1 - 1)) * 100 /
FIXED_1;
    unsigned long total_load = load * 100 + load_frac;
    int num_cpus = num_online_cpus();

    int percent = (int)(total_load / num_cpus);
    if (percent > 100) percent = 100;
```

```
    return percent;
}

static int get_memory_pressure_percent(struct aula_kbd *dev) {
    unsigned long all_events[NR_VM_EVENT_ITEMS];
    unsigned long current_pgmajfault;
    unsigned long delta;
    int percent;

    all_vm_events(all_events);
    current_pgmajfault = all_events[PGMAJFAULT];

    delta = current_pgmajfault - dev->prev_pgmajfault;
    dev->prev_pgmajfault = current_pgmajfault;

    if (delta > 100) percent = 100;
    else percent = (int)delta;

    return percent;
}

static int get_dirty_pages_percent(void) {
    unsigned long dirty = global_node_page_state(NR_FILE_DIRTY);
    struct sysinfo si;
    si_meminfo(&si);

    // dirty_ratio обычно около 20% от RAM, берем это за 100% шкалы
    unsigned long limit = (si.totalram * 20) / 100;
    if (limit == 0) limit = 1;

    unsigned long long percent_calc = ((unsigned long long)dirty * 100)
/ limit;
    if (percent_calc > 100) percent_calc = 100;

    return (int)percent_calc;
}
```

```
static void build_color_packet(unsigned char* buffer, int la_percent,
int mp_percent, int dp_percent) {
    struct Color c;
    memcpy(buffer, "\x06\x06\x00\x00\x01\x00\x80\x01", 8);

    int la_light = (la_percent == 0) ? 0 : (la_percent + 9) / 10;
    for (int i = 0; i < NUM_LA_KEYS; i++) {
        c = (i < la_light) ? lint_color(&la_gradient, i, NUM_LA_KEYS -
1) : (struct Color){0,0,0};
        set_key_color(buffer, load_avg_keys[i], c);
    }

    int mp_light = (mp_percent == 0) ? 0 : (mp_percent + 9) / 10;
    for (int i = 0; i < NUM_MP_KEYS; i++) {
        c = (i < mp_light) ? lint_color(&mp_gradient, i, NUM_MP_KEYS -
1) : (struct Color){0,0,0};
        set_key_color(buffer, mem_pressure_keys[i], c);
    }

    int dp_light = (dp_percent == 0) ? 0 : (dp_percent + 9) / 10;
    for (int i = 0; i < NUM_DP_KEYS; i++) {
        c = (i < dp_light) ? lint_color(&dp_gradient, i, NUM_DP_KEYS -
1) : (struct Color){0,0,0};
        set_key_color(buffer, dirty_pages_keys[i], c);
    }

    c = (struct Color) {255, 255, 255};
    for (int i = 0; i < NUM_BORDER_KEYS; i++)
        set_key_color(buffer, border_keys[i], c);
}

static void build_handshake_packet_1(unsigned char* buffer) {
    static const unsigned char data[] = {
        0x06, 0x84, 0x00, 0x00, 0x01, 0x00, 0x80
    };
```

```
    memcpy(buffer, data, sizeof(data));
}

static void build_handshake_packet_2(unsigned char* buffer) {
    static const unsigned char data[] = {
        0x06, 0x04, 0x00, 0x00, 0x01, 0x00, 0x80, 0x00, 0x00, 0x03,
0x03, 0x01, 0x00, 0x00, 0x04, 0x04,
        0x07, 0x01, 0x15, 0x20, 0x01, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x02, 0x01, 0x00, 0xff,
        0x0a, 0x00, 0x01, 0x00, 0x01, 0x00, 0x03, 0x01, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
        0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
        0xff, 0xff, 0x08, 0x40, 0x09, 0x47, 0x09, 0x47, 0x09, 0x47,
0x09, 0x47, 0x09, 0x47, 0x09, 0x47,
        0x09, 0x47, 0x09, 0x47, 0x09, 0x47, 0x09, 0x47, 0x09, 0x47,
0x09, 0x47, 0x09, 0x47, 0x09, 0x47,
        0x09, 0x47, 0x03, 0x37, 0x09, 0x37, 0x09, 0x37, 0x09, 0x37,
0x07, 0x47, 0x07, 0x47, 0x07, 0x44,
        0x07, 0x44, 0x07, 0x44, 0x07, 0x44, 0x07, 0x44, 0x07, 0x44,
0x07, 0x44, 0x04, 0x09, 0x04, 0x04,
        0x04, 0x04, 0x04, 0x04, 0x04, 0x04, 0x5a, 0xa5
    };
    memcpy(buffer, data, sizeof(data));
}

static void send_packet(struct usb_device *udev, unsigned char *data) {
    usb_control_msg(udev, usb_sndctrlpipe(udev, 0), 0x09, 0x21, 0x0306,
1, data, PACKET_BUFFER_SIZE, 500);
}

static int update_thread_func(void *data) {
    struct aula_kbd *dev = data;
    bool needs_update = false;
    int la_percent, mp_percent, dp_percent;
    unsigned long all_events[NR_VM_EVENT_ITEMS];
```

```
    all_vm_events(all_events);
    dev->prev_pgmajfault = all_events[PGMAJFAULT];

    while (!kthread_should_stop()) {
        la_percent = get_load_average_percent();
        mp_percent = get_memory_pressure_percent(dev);
        dp_percent = get_dirty_pages_percent();

        int current_la_level = (la_percent == 0) ? 0 : (la_percent + 9)
/ 10;
        int current_mp_level = (mp_percent == 0) ? 0 : (mp_percent + 9)
/ 10;
        int current_dp_level = (dp_percent == 0) ? 0 : (dp_percent + 9)
/ 10;

        needs_update = false;
        if (current_la_level != dev->last_la_level ||
            current_mp_level != dev->last_mp_level ||
            current_dp_level != dev->last_dp_level) {

            needs_update = true;
            dev->last_la_level = current_la_level;
            dev->last_mp_level = current_mp_level;
            dev->last_dp_level = current_dp_level;
        }

        if (needs_update) {
            send_packet(dev->udev, dev->handshake_packet_1);
            msleep(20);
            send_packet(dev->udev, dev->handshake_packet_2);
            msleep(20);

            memset(dev->color_packet, 0, PACKET_BUFFER_SIZE);
            build_color_packet(dev->color_packet, la_percent,
mp_percent, dp_percent);
```

```
            send_packet(dev->udev, dev->color_packet);
        }
        msleep(1000);
    }
    return 0;
}

static int aula_probe(struct usb_interface *interface, const struct
usb_device_id *id) {
    struct aula_kbd *dev;
    struct usb_device *udev = interface_to_usbdev(interface);

    dev = kzalloc(sizeof(*dev), GFP_KERNEL);
    if (!dev) return -ENOMEM;

    dev->handshake_packet_1 = kzalloc(PACKET_BUFFER_SIZE, GFP_KERNEL);
    dev->handshake_packet_2 = kzalloc(PACKET_BUFFER_SIZE, GFP_KERNEL);
    dev->color_packet = kzalloc(PACKET_BUFFER_SIZE, GFP_KERNEL);

    if (!dev->handshake_packet_1 || !dev->handshake_packet_2 || !dev-
>color_packet) {
        kfree(dev->handshake_packet_1); kfree(dev->handshake_packet_2);
kfree(dev->color_packet);
        kfree(dev);
        return -ENOMEM;
    }

    dev->udev = usb_get_dev(udev);
    usb_set_intfdata(interface, dev);

    dev->last_la_level = -1;
    dev->last_mp_level = -1;
    dev->last_dp_level = -1;

    build_handshake_packet_1(dev->handshake_packet_1);
    build_handshake_packet_2(dev->handshake_packet_2);
```

```
    dev->update_thread = kthread_run(update_thread_func, dev,
"aula_update_thread");
    if (IS_ERR(dev->update_thread)) {
        kfree(dev->handshake_packet_1); kfree(dev->handshake_packet_2);
kfree(dev->color_packet);
        kfree(dev);
        return PTR_ERR(dev->update_thread);
    }

    return 0;
}

static void aula_disconnect(struct usb_interface *interface) {
    struct aula_kbd *dev = usb_get_intfdata(interface);
    if (dev) {
        if (dev->update_thread) kthread_stop(dev->update_thread);
        kfree(dev->handshake_packet_1);
        kfree(dev->handshake_packet_2);
        kfree(dev->color_packet);
        usb_put_dev(dev->udev);
        kfree(dev);
    }
}

static const struct usb_device_id aula_table[] = { {
USB_DEVICE(AULA_VID, AULA_PID) }, {} };
MODULE_DEVICE_TABLE(usb, aula_table);
static struct usb_driver aula_driver = {
    .name = "aula_monitor",
    .probe = aula_probe,
    .disconnect = aula_disconnect,
    .id_table = aula_table,
};
```

```
static int __init aula_init(void) { return usb_register(&aula_driver);
}
static void __exit aula_exit(void) { usb_deregister(&aula_driver); }

module_init(aula_init);
module_exit(aula_exit);
```

Листинг 2 — Makefile

```
CURRENT = $(shell uname -r)
KDIR = /lib/modules/$(CURRENT)/build
PWD = $(shell pwd)
MOD_NAME := aula_monitor
obj-m := $(MOD_NAME).o
default:
	make -C $(KDIR) M=$(PWD) modules
clean:
	make -C $(KDIR) M=$(PWD) clean
test:
	sudo insmod $(MOD_NAME).ko && sudo dmesg | tail -n 20 && sudo
rmmod $(MOD_NAME) && echo "--- Module unloaded ---" && dmesg | tail -n
5
load:
	sudo insmod $(MOD_NAME).ko
reload:
	$(MAKE) -s unload || true
	$(MAKE) -s clean
	$(MAKE) -s default
	$(MAKE) -s load
unload:
	sudo rmmod $(MOD_NAME)
info:
	lsusb -t
unbind:
	sudo sh -c 'echo "3-2:1.1" > /sys/bus/usb/drivers/usbhid/unbind'
```